

Name: K RAJ BHARATH K RAJ BHARATH | Reg. No: RA2511003011548

Unit – 5 : Session – 7 : SLO – 1 | Implementation of MST — Prim's Algorithm

■ I. Pseudocode Fill-in-the-Blanks

1. Create a mstSet array to track VERTICES ALREADY INCLUDED IN THE MST.
2. Initialize the KEY array to track the minimum edge weight.
3. Set the key value for the starting vertex to 0.
4. For (V-1) times:
 - a. Pick the vertex u with the MINIMUM key value.
 - b. Include u in the MST.
 - c. For all adjacent vertices v: if v is not in MST and $\text{weight}(u,v) < \text{key}[v]$:
 - update $\text{key}[v]$; update $\text{PARENT}[v]$.

■ II. Complete the C Code Snippet

- `int minKey(int key[], bool mstSet[], int V) {`
- `int min = INT_MAX, min_index;`
- `for (int v = 0; v < V; v++)`
- `if (mstSet[v] == false && key[v] < min)`
- `min = key[v], min_index = v;`
- `return min_index;`
- `}`

■ III. Predict the Output

- Vertices 0, 1, 2; edges 0-1(2), 1-2(3), 0-2(1).
- Prim's from 0: pick 0-2 (1), then 0-1 (2) [cheaper than 1-2(3)].
- Total MST weight = $1 + 2 = 3$.

■ IV. Debug This Code

- Mistake: `mstSet[u] = false` — this marks u as NOT in the MST, so `minKey` can pick u again
- (infinite loop / wrong tree).
- Correction: `mstSet[u] = true`; (mark u as included in the MST).

■ V. Creative Reasoning

1. For dense graphs, the simple array-based Prim's runs in $O(V^2)$, which beats the $O(E \log V)$ heap version
 - when $E \sim V^2$; no heap overhead, and the matrix gives $O(1)$ edge access.
2. With all edge weights equal, every spanning tree has the same weight, so any tie-breaking is valid;
 - starting from different vertices can produce DIFFERENT valid MSTs (up to $n^{(n-2)}$ possible spanning trees
 - for a complete graph — e.g. 3 possible MSTs for a triangle of 3 vertices).